El mejor modelo

¿Por qué ganó LightGBM y qué implica para focalizar la pobreza?

Equipo 03: Julián Herrera, Andrés Silva y Valentina Vera

28 de octubre de 2026

MECA 4107 Big Data y Machine Learning para Economía Aplicada, Universidad de los Andes

Mejor modelo: LightGBM con 1.296 variables, 0,77 en Kaggle

Especificación

- Boosting de gradiente (LightGBM)
- Conjunto X_7: 1.296 variables del hogar y de sus miembros
- 31 hojas por árbol, tasa de aprendizaje 0,04, 748 rondas
- Penalidades L2 = 10 y L1 = 0,1
- Umbral 0,39, el que maximiza el F1 en validación cruzada

0,77

F1 en Kaggle (tabla pública)

F1 CV 0,768 · precisión 74,4% · recall 79,4%

Lo que muestra este deck

1. **Ganó por la información y por el algoritmo:** supera al mismo LightGBM con 43 variables (0,731) y a XGBoost con las mismas 1.296 (0,762).
2. **El arriendo y el trabajo formal concentran la predicción,** junto con otras fuentes de ingreso.
3. **Con el umbral del F1,** queda por fuera el 20,6% de los pobres y el 25,6% de la lista no es pobre. La agencia puede mover ese balance sin cambiar de modelo.

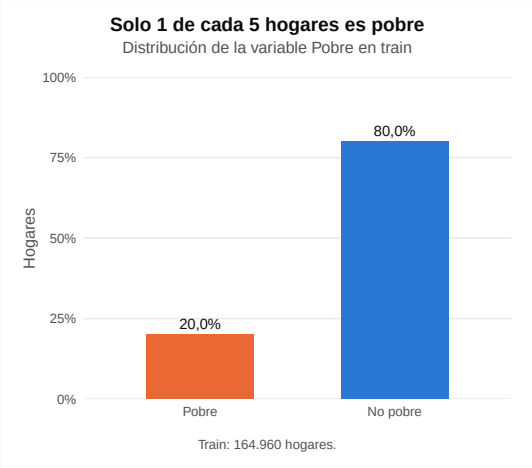
Datos: predecir la pobreza sin observar el ingreso

	Train	Test
Hogares	164.960	66.168
Personas	543.109	219.644
Hogares de Bogotá	10.567 (6,4%)	0
Hogares pobres	20,0%	A predecir

Fuente: DANE, Misión de Empalme (MESE).

- Un hogar es pobre si su ingreso per cápita está por debajo de la línea de pobreza. En test no está el ingreso: hay que predecir con lo que se asocia a él.
- Bogotá está en train pero no en test: revisamos más adelante si eso afecta al modelo.

Datos: el desbalance de clases



Clasificador ingenuo: nadie es pobre

Métrica	Todos no pobres
Accuracy	80,0%
F1	0
Sensibilidad (recall)	0%
Especificidad	100%

- Declarar a todos no pobres acierta el 80% de los hogares, pero no encuentra a ningún pobre.
- Por eso evaluamos con F1, que solo sube si se encuentran pobres sin llenar la lista de no pobres.

Predictores: 1.296 variables con una lógica económica

Qué aproxima el ingreso del hogar

- **Vivienda:** arriendo, tenencia y hacinamiento.
- **Trabajo y protección:** ocupación, formalidad, salud y prestaciones.
- **Capital humano:** educación y edades de los miembros.
- **Otros recursos:** pensiones, rentas y transferencias.
- **Territorio:** ciudad, zona y departamento.

Cómo se construyen desde las personas

- **Composición:** conteos y proporciones de miembros por categoría.
- **Agregados:** medias, máximos, mínimos, sumas y dispersión.
- **Indicadores relativos:** personas por cuarto, miembros por ocupado y arriendo por miembro.
- **Perfiles:** jefe, cónyuge y los tres miembros de mayor edad.

Ejemplo: una pregunta de educación da la educación del jefe, el promedio del hogar y el máximo de los ocupados. Como el ingreso de un hogar depende de cuántos trabajan, en qué y entre cuántos se reparte, describir a cada miembro aporta información.

Entrenamiento y elección de hiperparámetros

Decisión	Qué hicimos
Algoritmo	Árboles en secuencia: cada uno corrige los errores de los anteriores. Maneja datos faltantes, que X_7 tiene.
Validación	5 folds, los mismos de todos los modelos; F1 fuera de muestra.
Hiperparámetros	31 hojas, tasa 0,04, 748 rondas, L2 = 10, L1 = 0,1, mínimo 100 hogares por hoja, 85% de variables y hogares por ronda.
Cómo se eligieron	Exploración inicial, verificada después con una grilla de 36 combinaciones (siguiente slide). La configuración es la misma en X_5, X_6 y X_7, para que la diferencia entre conjuntos sea solo de variables.
Umbral	0,39: el que maximiza el F1 con las predicciones fuera de muestra.
Desbalance	Se corrige con el umbral. Rebalancear las clases movió el F1 menos de 1 punto en los demás algoritmos.

El umbral se elige con las mismas predicciones con las que se mide el F1, así que el F1 de validación cruzada puede ser algo optimista; el 0,77 de Kaggle, con hogares nuevos, lo confirma.

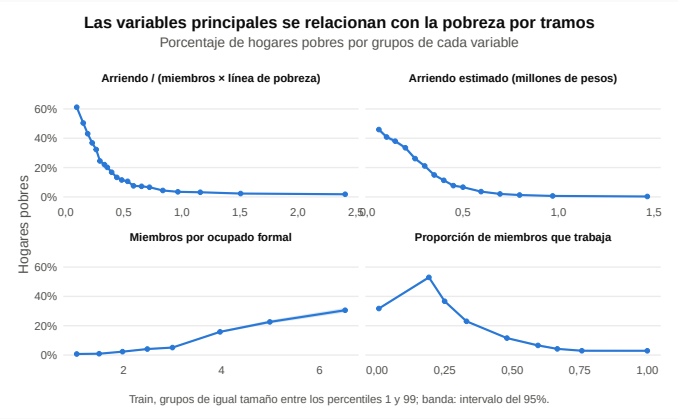
Búsqueda de hiperparámetros: la configuración está entre las mejores

Posición	Hojas	Tasa	Mínimo por hoja	Rondas	F1	
1	31	0,02	300	1.496	0,7320	
2	31	0,02	100	1.496	0,7320	
3	31	0,02	20	1.496	0,7318	
4	31	0,04	20	748	0,7318	
5	63	0,02	20	1.496	0,7315	
6	31	0,04	300	748	0,7314	
7	63	0,02	100	1.496	0,7312	
8	31	0,04	100	748	0,7310	Elegida

Las 8 mejores de 36 combinaciones de hojas por árbol (15 a 127), tasa de aprendizaje (0,02 a 0,1) y mínimo de hogares por hoja (20 a 300). Validación cruzada con X_5 y los mismos 5 folds.

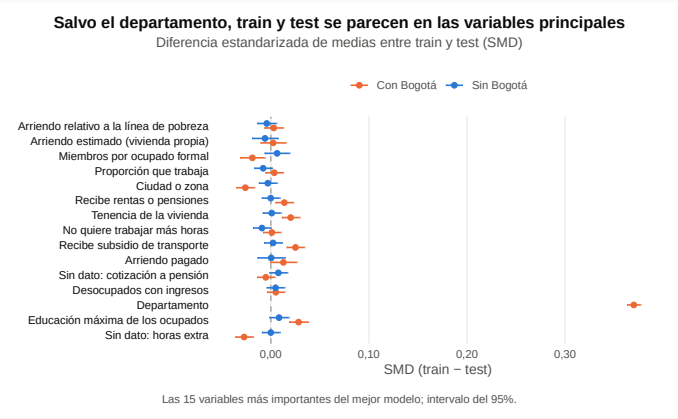
- **Qué hicimos:** la configuración se eligió con una exploración inicial y luego se verificó con esta grilla, frente a otras 35 combinaciones.
- **Resultado:** las 36 quedan entre 0,725 y 0,732. La nuestra está en el puesto 8, a 0,1 puntos de la mejor.
- **Por qué 0,1 no importa:** con la misma configuración, el F1 cambia de 0,729 a 0,735 según el fold (0,6 puntos). Una diferencia de 0,1 es menor que esa variación: no sabemos cuál es mejor con hogares nuevos.
- **Lo que sí muestra:** 31 hojas es la mejor complejidad (127 sobreajusta), y los hiperparámetros mueven el F1 menos de 1 punto, frente a 3,7 de las variables.

¿Por qué funcionó? Las variables principales actúan por tramos



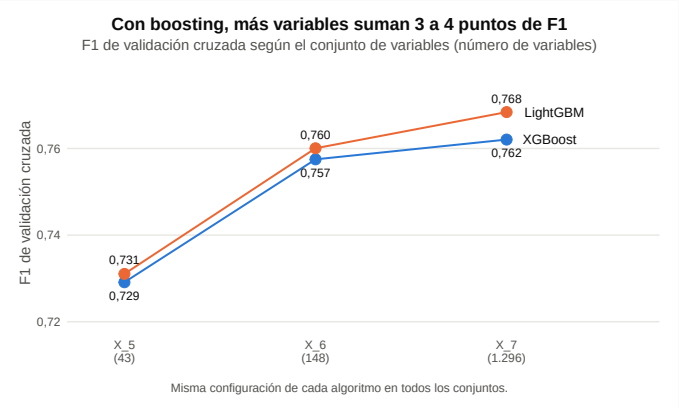
- La pobreza cae rápido con el arriendo y luego casi no cambia.
- Sube con los miembros por ocupado formal: el ingreso formal se reparte entre más personas.
- No es monótona con la proporción que trabaja.
- Son relaciones por tramos, que un modelo de árboles capta sin especificarlas.

¿Por qué funcionó? Entrenar con Bogotá no sesgó el modelo



- **Pregunta:** Bogotá es el 6,4% de train y no está en test. ¿Entrenar con ella aleja a train de test?
- **Prueba:** diferencia de medias entre train y test, en desviaciones estándar (SMD), para las 15 variables principales.
- **Resultado:** todas cerca de 0, con y sin Bogotá, salvo el departamento (Bogotá no existe en test).
- **Decisión:** conservar Bogotá: más datos sin cambiar lo que usa el modelo.

¿Por qué funcionó? Más información y un mejor algoritmo



- **Información:** con la misma configuración, pasar de 43 a 1.296 variables suma 3,7 puntos de F1.
- **Algoritmo:** con las mismas variables, LightGBM supera a XGBoost (0,762) y, con X_5, a Random Forest y Elastic Net.
- La mayor parte de la ganancia llega con X_6: el detalle adicional de X_7 suma menos de 1 punto.

¿Por qué funcionó? Conclusiones

1. Más información sobre el hogar

Pasar de 43 a 1.296 variables suma 3,7 puntos de F1: es la mayor ganancia.

2. Un algoritmo que capta relaciones por tramos

El boosting aprende umbrales e interacciones como las del arriendo y el trabajo formal sin que haya que especificarlas.

3. El umbral resolvió el desbalance

Elegir el umbral que maximiza el F1 compensó que solo el 20% sea pobre, sin rebalancear las clases.

4. Bogotá sumó datos sin sesgar

Las variables principales se distribuyen igual en train y test con o sin Bogotá.

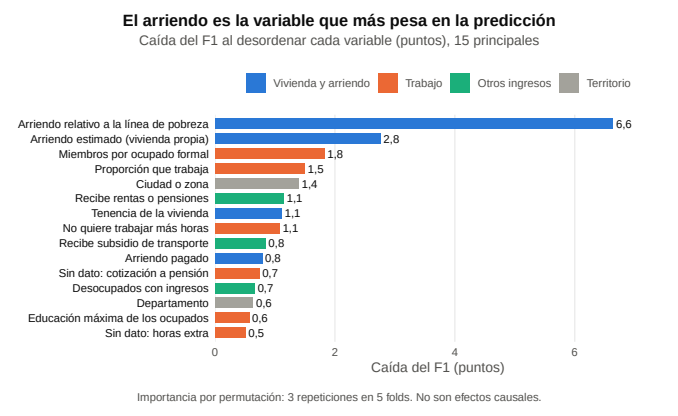
El reto de la interpretabilidad

Un LightGBM con 1.296 variables y 748 rondas de árboles no se puede leer directamente: no tiene un coeficiente por variable.

- **Importancia por permutación:** cuánto cae el F1 si se desordena una variable, es decir, cuánto depende la predicción de ella.
- **Lectura económica:** relacionar las variables principales con lo que determina el ingreso del hogar y con lo que se puede preguntar en campo.

Podemos interpretar la predicción, sin atribuirle efectos causales.

Importancia: el arriendo es lo que más pesa



- El arriendo relativo a la línea de pobreza es, por mucho, la variable principal: aproxima el valor de la vivienda frente al costo de vida.
- Le siguen el trabajo formal (miembros por ocupado formal, proporción que trabaja) y otras fuentes de ingreso (rentas, pensiones, subsidios).
- Miden cuánto depende la predicción de cada variable, no efectos causales.

Diagnóstico: qué acierta y en qué se equivoca

Matriz de confusión (umbral 0,39)

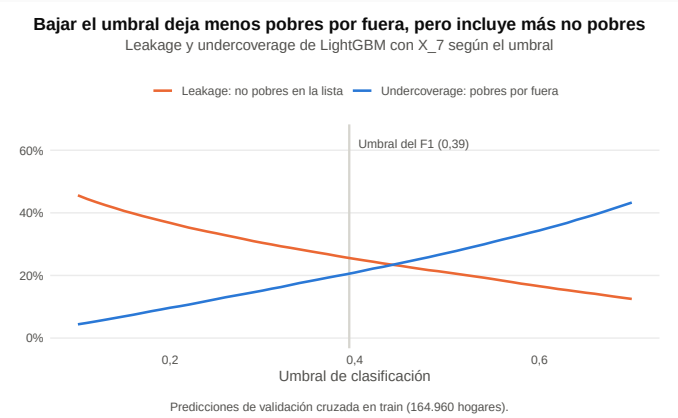
	Pobre	No pobre
Predicho pobre	26.236	9.027
Predicho no pobre	6.788	122.909

Métrica	Valor
Precisión	74,4%
Recall	79,4%
F1	0,768
AUC	0,957
Accuracy	90,4%

- De cada 4 hogares que el modelo señala como pobres, 3 lo son (precisión 74,4%).
- Encuentra a 4 de cada 5 hogares pobres (recall 79,4%); quedan por fuera 6.788.

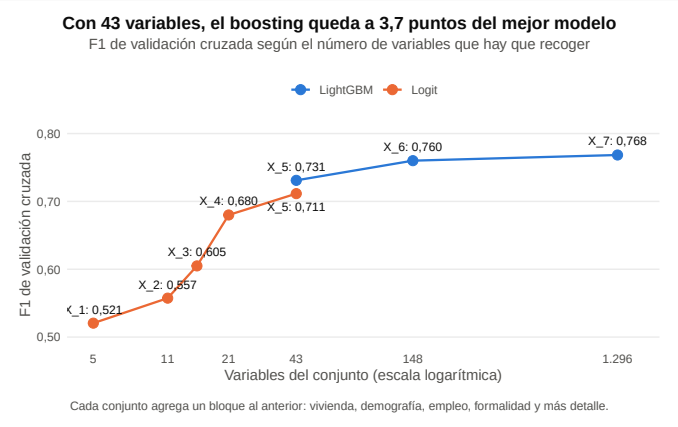
Predicciones de validación cruzada en train: cada hogar, con un modelo que no lo usó para entrenar.

¿Qué balance entre leakage y undercoverage prefiere la política?



- El F1 pesa igual los dos errores: con 0,39 quedan por fuera 6.788 pobres y entran 9.027 no pobres.
- Con umbral 0,30 el F1 casi no cambia (0,765), pero los pobres por fuera bajan del 20,6% al 15,2%, a cambio de más leakage (30,4%).
- Si excluir a un pobre cuesta más que incluir a un no pobre, conviene un umbral más bajo que el del F1.

¿Se puede aplicar con una encuesta corta?



- Con 43 variables (vivienda, composición, empleo y arriendo), el boosting logra 0,731: 3,7 puntos menos que con 1.296.
- Fácil de verificar en una visita: vivienda, quién vive y quién trabaja.
- Difícil: el arriendo de una vivienda propia es una estimación, y las ayudas o pensiones están cerca del ingreso que el hogar podría ocultar.

Recomendación: encuesta corta con boosting, y verificar en campo las variables de vivienda y trabajo.

La encuesta corta: qué se pierde y qué se gana

	X_7 (1.296 variables)	X_5 (43 variables)
Pobres encontrados	26.236	27.088
Pobres por fuera (undercoverage)	6.788	5.936
No pobres en la lista (leakage)	9.027	13.996
Hogares en la lista	35.263	41.084
Precisión	74,4%	65,9%
Recall	79,4%	82,0%
F1	0,768	0,731

- **Se pierde precisión:** la lista crece en 5.821 hogares, de los cuales 4.969 no son pobres; la precisión baja de 74,4% a 65,9%.
- La cobertura no se pierde: X_5 encuentra 852 pobres más (recall de 82,0% frente a 79,4%).
- **Se gana una encuesta fácil de recoger:** 43 variables en lugar de 1.296, verificables en una visita.
- El modelo sigue siendo bueno: F1 de 0,73, mejor que cualquier otro algoritmo con esas variables.

¡Gracias!